

5. Polymorphism

What

- Means many forms
- The ability of a single function or Operator to work in different ways based on the object
- Allows the object to have several different forms and behaviors

Type

- Compile Time Polymorphism (Static)
- Runtime Polymorphism (Dynamic)

Static Polymorphism

- Compile time
- types
 - Function Overloading
 - Operator Overloading

Function Overloading

- functions with the same name
 - Differ in argument *type* or their *number*
 - Differ in *Return* type

C++

Folded Code

```
1 int sum(int x,int y){
2     return x+y;
3 }
4
5 double sum(double x,double y){
6     return x+y;
7 }
```

Operator Overloading

C++

Operator overloading basic SYNTAX

```

1  returnType operator <operatorType>(args){
2      // something
3  }

```

C++ Operator overloading

```

1  class ComplexNum{
2      public:
3          ComplexNum operator+(const ComplexNum obj)
4          {
5              //this → lhs
6              //obj → rhs
7              return ComplexNum(this→real+obj→real,
8                              this→complex+obj→complex);
9          }
10
11
12  };

```

- LHS - this

- RHS - argument

Operators That Can Be Overloaded

Category	Operators
Arithmetic	+, -, *, /, %, ++, --
Relational	==, !=, <, >, <=, >=
Logical	&&, , !
Bitwise	&, , ^, ~, <<, >>
Assignment	=, +=, -=, *=, /=, %=, &=, =, ^=, <<=, >>=
Increment/Decrement	++, --
Subscript	[]
Function Call	()
Pointer-like	->, *, &
Memory Management	new, new[], delete, delete[]
Comma	,
Stream	<<, >>

Operators That Cannot Be Overloaded

Operator	Meaning
::	Scope resolution
.	Member access
.*	Pointer-to-member
?:	Ternary operator
sizeof	Size operator
typeid	Type information
alignof	Alignment requirement

Important Rules

Rule	Description
At least one operand	Must be a user-defined type
Cannot modify precedence	Operator priority stays same
Cannot modify associativity	Evaluation order stays same
Must be member functions	=, [], (), ->

Defining my own class using all the concepts used

C++ Complex Number

```

1  class Complex{
2  private:
3      int real;
4      int img;
5  public:
6
7      Complex(); // default constructor
8      Complex(int r, int i); // parameterized constructor
9
10     Complex operator+(const Complex &rhs);
11     Complex operator-(const Complex &rhs);
12
13     Complex operator*(const Complex &rhs);
14     Complex operator*(const int rhs);
15
16     Complex operator/(const Complex &rhs);
17     Complex operator/(const int rhs);
18
19     void operator+=(const Complex &rhs);
20     void operator-=(const Complex &rhs);

```

```

21
22     void operator*=(const Complex &rhs);
23     void operator*=(const int rhs);
24
25     void operator/=(const Complex &rhs);
26     void operator/=(const int rhs);
27
28     float rationalize(const Complex &);
29     Complex complexConjugate(const Complex &rhs);
30
31     // 2+5i += 3+2i
32     // 2+5i *= 3+2i
33     // 2+5i *= 3
34     //
35     void display();
36 };
37
38 Complex::Complex() : real{-1}, img{-1} {}
39 Complex::Complex(int r, int i) : real{r}, img{i} {}
40 3
41 Complex Complex::operator+(const Complex &rhs){
42     return Complex(this->real + rhs.real,
43         this->img + rhs.img);
44 }
45 Complex Complex::operator-(const Complex &rhs){
46     return Complex(this->real - rhs.real,
47         this->img - rhs.img);
48 }
49 Complex Complex::operator*(const Complex &rhs){
50     return Complex(
51         (this->real) * (rhs.real) - (rhs.img) * (this->img),
52         (this->img) * (rhs.real) + (this->real) * (rhs.img));
53 }
54
55 Complex Complex::operator*(const int rhs){
56     return Complex(rhs * (this->real),
57         rhs * (this->img));
58 }
59
60 Complex Complex::operator/(const Complex &rhs){
61     Complex ans = Complex();
62     ans = (*this) * complexConjugate(rhs);
63     return ans / rationalize(rhs);
64 }
65
66 Complex Complex::operator/(const int rhs){
67     return Complex((this->real) / rhs,
68         rhs * (this->img) / rhs);
69 }

```

```

70
71 void Complex::operator+=(const Complex &rhs){
72     this→real += rhs.real;
73     this→img += rhs.img;
74 }
75
76 void Complex::operator-=(const Complex &rhs){
77     this→real -= rhs.real;
78     this→img -= rhs.img;
79 }
80
81 void Complex::operator*=(const Complex &rhs){
82     Complex asn = *this * rhs;
83     this→real = asn.real;
84     this→img = asn.img;
85 }
86
87 void Complex::operator*=(const int rhs){
88     this→real *= rhs;
89     this→img *= rhs;
90 }
91
92 void Complex::operator/=(const Complex &rhs){
93     Complex asn = *this / rhs;
94     this→real = asn.real;
95     this→img = asn.img;
96 }
97
98 void Complex::operator/=(const int rhs){
99     this→real /= rhs;
100    this→img /= rhs;
101 }
102
103 float Complex::rationalize(const Complex &obj){
104     return sqrt(
105         (obj.img) * (obj.img) +(obj.real) * (obj.real));
106 }
107
108 Complex Complex::complexConjugate(const Complex &rhs){
109     return Complex(rhs.real, -rhs.img);
110 }
111
112 void Complex::display(){
113     printf("(%d,%d)\n", this→real, this→img);
114 }
115
116 int main(int argc, char const *argv[]){
117     Complex A(2,3);
118     A += Complex(3,4);

```

```
119     A.display();
120     return 0;
121 }
```

Runtime Polymorphism

Problem : 'Early binding'

- (also called **static binding** or **compile-time binding**)
- mechanism where the compiler resolves method calls, variable types, and function references **at compile time**
- before the program runs.

Context

C++

Example

```
1  class Animal {
2  public:
3      void speak() {
4          cout << "Animal speaks" << endl;
5      }
6  };
7
8  class Dog : public Animal {
9  public:
10     void speak() {
11         cout << "Dog barks" << endl;
12     }
13 };
14
15 int main() {
16     Animal* a = new Dog();
17     a->speak(); // Output: "Animal speaks" ← Early binding!
18 }
```

Note

Here, since `speak()` is **not virtual**, the compiler binds the call to `Animal::speak()` based on the **declared type** (`Animal*`), not the actual object (`Dog`). This is early binding

Virtual Keyword

- The **virtual** keyword delays the compiler's decision until **runtime**.

C++ Example

```

1  class Animal {
2  public:
3      virtual void speak() {
4          cout << "Animal speaks" << endl;
5      }
6  };
7
8  class Dog : public Animal {
9  public:
10     void speak() {
11         cout << "Dog barks" << endl;
12     }
13 };
14
15 int main() {
16     Animal* a = new Dog();
17     a->speak(); // Output: "Dog Barks"
18                //← Early binding issue fixed
19 }

```

	Decision Time
Without virtual	Compiler decides now (compile time)
With virtual	Compiler decides later (runtime)

Override

- The **override** keyword is used in a child class to **explicitly tell the compiler** that you are intentionally overriding a **virtual** function from the parent class.
- Synopsis
 - override** = "Hey compiler, I am intentionally replacing my parent's virtual function. Please verify that."

C++ without override

```

1  class Animal {
2  public:
3      virtual void speak() { cout << "Animal"; }
4  };
5
6  class Dog : public Animal {
7  public:

```

```

8     void speak() {
9         cout << "Dog"; // works, but compiler won't verify
10    }
11 };

```

C++ with override

```

1  class Animal {
2  public:
3      virtual void speak() { cout << "Animal"; }
4  };
5
6  class Dog : public Animal {
7  public:
8      void speak() override{
9          cout << "Dog"; // compiler verifies this
10     }
11 };

```

🔗 **virtual** gives permission to override. **override** confirms you are doing it correctly.

More on UP-CASTING & DOWN-CASTING

- These are about **converting object references** between parent and child types in inheritance.

UP - CASTING

📌 **Treating a child class object as a parent class type.**

- Goes **UP** the inheritance hierarchy (Child → Parent)
- **Automatic / Implicit** — compiler does it safely
- **Always safe** — every child IS a parent

```

1  class Animal {
2  public:
3      virtual void speak() { cout << "Animal"; }
4  };
5
6  class Dog : public Animal {
7  public:
8      void speak() override { cout << "Dog"; }
9      void fetch() { cout << "Fetching!"; }
10 };

```

```

11
12 int main() {
13     Dog d;
14     Animal* a = &d;    // ✓ Upcasting – automatic, no casting needed
15     a->speak();         // Output: "Dog" (virtual still works!)
16 }

```

What You LOSE with Upcasting

```

1 a->fetch();    // ✗ ERROR! – fetch() is not in Animal

```

You **lose access** to child-specific methods because the compiler only sees the **parent type**.

Down-casting

Treating a parent class pointer/reference BACK as a child class type.

- Goes **DOWN** the hierarchy (Parent → Child)
- **Manual / Explicit** — you must do it yourself
- **Risky** — not always safe

```

1 int main() {
2     Animal* a = new Dog();    // Upcasted first
3
4     Dog* d = (Dog*)a;        // Downcasting – manual cast
5     d->fetch();              // ✓ Now you can access fetch()
6 }

```

Safe Downcasting with `dynamic_cast`

Raw casting is dangerous. Use `dynamic_cast` to cast **safely**:

```

1 int main() {
2     Animal* a = new Dog();
3
4     Dog* d = dynamic_cast<Dog*>(a);    // Safe downcast
5
6     if (d != nullptr) {
7         d->fetch();    // ✓ Safe to use
8     } else {
9         cout << "Cast failed!";    // Won't crash
10    }
11 }

```

Why `dynamic_cast` is Safer

```

1  Animal* a = new Animal();           // NOT a Dog!
2
3  Dog* d = dynamic_cast<Dog*>(a);      // returns nullptr – safe ✓
4  Dog* d = (Dog*)a;                   // Undefined behavior – dangerous ✗

```

Visual Picture

```

1      Animal      ← Parent (Base)
2      |
3      Dog         ← Child (Derived)
4
5
6  Upcasting:   Dog → Animal   (going UP)   ✓ Automatic, Safe
7  Downcasting: Animal → Dog   (going DOWN) ⚠ Manual, Risky

```

Side by Side Comparison

	Upcasting	Downcasting
Direction	Child → Parent	Parent → Child
How	Automatic	Manual
Safety	Always safe	Can fail
Access	Loses child methods	Regains child methods
Cast needed	No	Yes (<code>dynamic_cast</code>)

Connecting Everything So Far

```

1  virtual    → delay decision to runtime
2  override   → confirm you are replacing virtual function
3  upcasting   → treat Dog AS Animal (safe, automatic)
4  downcasting → treat Animal BACK AS Dog (risky, manual)

```

One Line Summary

📋 **Upcasting is widening your view (safe). Downcasting is narrowing your view (risky – verify first).**

Final Keyword

- used to stop further over-riding or further inheritance
 - synopsis :- > `final` = "This is the end. No one can go beyond this point."

Stopping inheritance

C++

Final keyword

```
1 class Animal final {    // No one can inherit from Animal
2 public:
3     void speak() {
4         cout << "Animal";
5     }
6 };
7
8 class Dog : public Animal {    // ❌ COMPILE ERROR!
9 };
```

🔗 Once a class is marked `final`, **no child class can inherit** from it.

On Virtual Function

- prevents over-riding of the `virtual` function

C++

Virtual function and final

```
1 class Animal {
2 public:
3     virtual void speak() {
4         cout << "Animal";
5     }
6 };
7
8 class Dog : public Animal {
9 public: void speak() final {
10     cout << "Dog";
11 } // No further overriding allowed
12 };
13 class Labrador : public Dog {
14 public: void speak() override {
15     cout << "Labrador";
16 } // ❌ COMPILE ERROR! };
```